Composition Is Where Obfuscation Robustness Breaks

ANONYMOUS AUTHOR(S)

Fine-tuning a code model on obfuscated programs raises its accuracy on those programs by roughly nineteen points. We ask what that number is made of: competence at reading the program, or competence at the surface a particular transform leaves behind. We separate the two with tests that share no machinery — whether the gain survives being stacked with a transform family the model has never seen, and whether it survives the task being run backwards. The field's existing answers, inference-time routing, weight-space merging, and training on a breadth of transforms, fail both. Breadth composes on stacks built from what it saw and loses that advantage the moment an unseen family enters, a gap significant on every one of six models across four lineages; and every method that fits the forward task buys its gain by losing backward competence on the very same programs. Three further instruments locate what is learned instead: the surface a transform leaves, which is exactly what an obfuscator is free to change. That diagnosis names its own remedy. We anchor the model to the one thing a semantics-preserving transform leaves invariant, its own behaviour on the unobfuscated parent of each program, through a paired-consistency objective that trains on exactly the rows breadth trains on. Re-running the same two tests, the anchored model is the only one whose advantage *grows* as input diverges from training, rising above the clean-code control on stacks containing an unseen family, and the only one whose forward gain costs nothing backwards. Its repair holds on six of eight models and reverses on one, and on the unseen family alone it is as robust as clean-code tuning and no more. Adaptation to obfuscated code buys recombination of what the data covered; anchoring recovers what that costs; neither buys invariance.

1 Introduction

Code obfuscation is widely used to make software difficult to inspect while preserving its intended functionality. In benign settings, developers employ obfuscation to protect intellectual property, implementation details, and proprietary business logic. In adversarial settings, however, malware authors routinely exploit the same techniques to conceal malicious behavior and frustrate program analysis and reverse engineering. Common transformations include identifier renaming, control-flow restructuring, dead-code insertion, expression rewriting, call indirection, etc. Although such transformations are intended to preserve program semantics, they can substantially obscure how those semantics are expressed in the source code, thereby increasing the difficulty of understanding the behavior of malware code [2, 3, 10, 11].

This difficulty is well established for human analysts. Nguyen *et al.* [7], for example, showed that code obfuscation significantly reduces humans' accuracy in reasoning about program outputs. Such findings are particularly important for malware analysis, where analysts must often infer the behavior of code whose surface structure has been deliberately engineered to resist comprehension.

Large language models (LLMs) offer a promising alternative for automating such reasoning. Models have demonstrated strong capabilities in code generation, summarization, vulnerability detection, and program understanding [1, 4–6, 8]. However, most evaluations are performed on naturally written programs, where meaningful identifiers, familiar control structures, and common coding idioms provide abundant lexical and structural cues. Obfuscated programs remove or distort many of these cues while retaining the underlying computation, creating a useful stress test of whether an LLM actually reasons about program behavior or instead depends on surface regularities.

Recent studies suggest that this distinction matters. Rong *et al.* [**?**] showed that semantics-preserving obfuscations can cause substantial degradation in LLM program reasoning, including

program-output prediction. Similarly, Nikiema *et al.* [9] reported a statistically significant decline in model performance as the complexity of obfuscation increases. Together, these findings indicate that this limitation is especially problematic for security applications: an AI-assisted malware analyzer should not cease to understand a malicious computation merely due to obfuscation.

An important challenge, however, has received considerably less attention: *real obfuscated programs rarely contain only one transformation.* Obfuscators can combine multiple techniques so that, for example, renamed identifiers are embedded inside flattened control flow, rewritten expressions, and additional irrelevant code. Such *stacked obfuscation* creates a fundamentally harder deployment setting. Success on each transformation independently does not necessarily imply success when those transformations occur together. Consequently, simply demonstrating that a model can be adapted to individual obfuscations is insufficient. For practical malware analysis, the learned capability must also *compose*.

This observation naturally suggests several ways to compose knowledge learned from individual transformations. One could train a specialist for each obfuscation and use a *router* to select the appropriate specialist at inference time. Alternatively, one could combine the specialists in parameter space through *model merging*. A third, and seemingly more direct, strategy is to increase *training breadth* by fine-tuning a single model on examples from many individual obfuscations, hoping that knowledge learned separately will recombine when transformations are stacked. We systematically evaluate these three strategies and find that none provides the desired robustness. The learned router performs no better than a random routing policy; merging specialized models performs at or below the clean-code-tuned control; and broader training, although effective on stacks assembled from transformations represented during training, introduces a substantial performance penalty on an unseen obfuscation family. Thus, broader exposure can improve recombination within the training distribution while simultaneously making the model *less* robust outside that distribution.

Our analysis reveals why. The apparent robustness obtained from obfuscation-specific adaptation does not primarily arise from learning the semantics that remain invariant under transformation. Instead, models learn regularities associated with the *surface artifacts* left by particular obfuscators. Evidence for this behavior appears across multiple analyses: adaptation transfers poorly between distinct mechanisms within the same obfuscation family; the benefit of broad training is substantially stronger when recognizable identifier cues remain available; and improvements disappear when the same programs are queried in a different reasoning direction. Thus, stacking exposes a deeper problem than merely insufficient training coverage: obfuscation-specific fine-tuning can teach the model how particular transformations *look* without teaching it to consistently recover the behavior that those transformations preserve.

This diagnosis suggests a different principle for robust adaptation. Rather than attempting to compose transformation-specific knowledge, we anchor learning to something that is invariant across all such transformations: the model's behavior on the corresponding *unobfuscated program*. For every obfuscated training program x_{obf}, we retain its clean parent x_{clean}. A teacher model, tuned for reasoning over clean code, processes x_{clean}, while the student processes x_{obf}. In addition to the ordinary task loss, we minimize the KL divergence between their output distributions:

$$\mathcal{L} = \mathrm{CE}(y \mid x_{\text{obf}}) + \lambda\, D_{\mathrm{KL}}\big(p_{\text{teacher}}(\cdot \mid x_{\text{clean}}) \,\|\, p_{\text{student}}(\cdot \mid x_{\text{obf}})\big)\,. \tag{1}$$

The first term exposes the student to obfuscated code, whereas the second explicitly encourages it to preserve the behavioral response of a model reasoning over the semantics-equivalent clean program. Importantly, this objective does not require recognizing which obfuscations are present, selecting among specialists, merging transformation-specific parameters, or recovering the original source code. Instead, it changes the learning target itself: rather than anchoring adaptation to the

surface patterns of individual transformations, it anchors the student to the clean-code behavior that those transformations should preserve.

Our empirical results demonstrate the advantage of this *paired-consistency anchoring*. In contrast to routing, model merging, and breadth-only training, the KL-based objective retains the benefits of learning from diverse obfuscations without paying the corresponding penalty on an unseen obfuscation family. At the 7B scale, the anchored model improves over breadth training by 4.59 percentage points on the unseen family while introducing no degradation on clean code. Additional stress tests show that the resulting model continues to perform well under deeper compositions and transfers to a second programming language. At the same time, our analysis carefully bounds the claim: the method does not create a new ability to solve arbitrary unseen obfuscations beyond that of the clean-code teacher. Rather, it prevents obfuscation adaptation from destroying capabilities the model already possesses. This distinction is important: robust reasoning over obfuscated programs requires not only acquiring adaptation gains, but also preserving semantic competence as the surface form of the program changes.

Contributions. This paper makes the following main contributions:

1. A two-test diagnostic for whether obfuscation adaptation generalises or memorises, and its verdict on the existing composition strategies. We ask whether competence learned on individual semantics-preserving transformations survives leaving the distribution it was fit to, in two independent ways: being stacked with a transformation family the model has never seen, and being asked backwards (given a return value, produce a call that yields it). Inference-time routing, weight-space merging, and broad multi-transformation training all fail both. Breadth's advantage on stacks of seen transformations vanishes once an unseen family enters — a gap significant on six of six models across four lineages — and every method that fits the forward task buys its gain by losing backward competence on the same programs.

2. An empirical characterisation of what adaptation learns instead. With complementary analyses across transformation mechanisms, identifier dependence, and confidence behaviour, we show that obfuscation adaptation predominantly learns the surface regularities a transformation leaves behind rather than an invariance to semantics-preserving change. Because that surface is exactly what an obfuscator is free to alter, the diagnosis names its own remedy.

3. A clean-behaviour anchoring objective, derived from that diagnosis. We introduce a paired-consistency objective that anchors an obfuscated-code student to a clean-code-tuned teacher reading the unobfuscated parent of the same program — the one thing a semantics-preserving transformation leaves invariant. The objective trains on exactly the rows breadth training uses, so breadth is its data-matched control. Across eight models and four lineages it removes breadth's unseen-family degradation on six and reverses on one, and its clean-code cost is absent on five; we report both qualifiers rather than the unconditional claim.

4. The same two tests, re-run on the anchored model, with the bound stated. On a new divergence ladder of stacks that contain an unseen transformation family — a stimulus absent from prior work — the anchored model's advantage *grows* as the input diverges from training, rising above the clean-code control where breadth falls, and it is the only supervised-fine-tuning arm whose forward gain costs nothing backwards. It does not acquire backward competence, and on the unseen family alone it is as robust as clean-code tuning and no more: anchoring repairs what adaptation breaks rather than creating new invariance.

2 Background

3 Setup

Because negative-leaning results are frequently attacked at their experimental foundation, our setup is designed to be highly rigorous. Our evaluation relies on a structured ladder consisting of the normalised original code (L0) and five single transforms (e.g., L1b adversarial renaming, S1 control-flow flattening). These are strictly isolated as single-transforms from the L0 parent and never stacked during training. Composites are deliberately placed in a separate namespace so that any transfer result can be reliably attributed to a single mechanism.

To test true invariance, we employ a held-out family (H1) and its trainable proxy (X1). H1 is strictly quarantined behind four independent enforcement layers, meaning every unseen-family measurement reported in our exploration relies on X1, which reproduces H1's difficulty with extreme fidelity ($r = 0.9992$).

Our correctness machinery utilizes program-level splits and enforces a strict normalised exact match criterion, explicitly rejecting containment matching to avoid false positives. We support these measurements with robust statistics, primarily program-clustered bootstraps with 2,000 resamples to account for correlated input cases. Finally, we establish an important measurement finding regarding model evaluation: untuned format failure fundamentally measures a model's adherence to the prompt contract, rather than its inherent capability. Screening base models by untuned format failure is therefore structurally unsound.

4 RQ1: Do current adaptation methods generalise, or memorise?

Adapting a code model to obfuscated input works, and works easily. Tuning on output prediction over obfuscated programs gains roughly **19 points** over the untuned model on the transforms it was trained on. The question this paper asks is what that number is made of. Adaptation can raise forward accuracy two ways: by teaching the model to *read* the program better, or by teaching it the surface a particular transform leaves behind. Only the first deserves to be called robustness.

We separate them with two tests. Both ask the same question — does the competence survive leaving the distribution it was fit to? — and they share no machinery.

- **Composition.** Real obfuscators stack transforms. Does competence on single transforms survive being stacked with a transform family the model has never seen?
- **Reversal.** Does it survive the task being run backwards — given a return value, produce a call that yields it? No adapter is ever trained on this direction, so any backward gain is transferred program understanding rather than task fitting.

We apply both to the field's existing answers: inference-time routing over per-transform specialists, weight-space merging of those specialists, and simply training on a breadth of transforms. These are baselines, not proposals. They fail both tests, and they fail in the same direction.

4.1 The composition test

Routing and merging do not compose at all. A learned router over per-transform specialists beats a *random* gate by +0.0000 [−0.0081, +0.0081]: the mixture is worth something and the routing is worth nothing. Task-vector merging (TIES, DARE-TIES, DARE-linear) lands at or below the clean-code-only control, −3.13 [−4.78, −1.40], while the specialists being merged contribute +2.47 [+1.32, +3.70] individually.

Training breadth is the one that appears to work, and it is the interesting case. It composes on stacks built from transforms it saw — and the advantage evaporates the moment a stack contains a family it did not. We state this as a *difference* rather than as two separate levels, because one

interval being positive while another is negative is not a test of the difference between them, and on four of the six models the unseen-side interval straddles zero on its own.

Table 1. **RQ1: the dissociation, measured within model on one program set.** Breadth (`mono_all`) against the clean-code control (`tuned_L0`) on stacks built from *seen* transforms and on stacks containing an *unseen* family. The *difference* is the measured quantity; the two levels are context, and on most models the unseen-side interval straddles zero on its own. Both halves are paired on the 319 programs common to every cell of both groups. Bold intervals exclude zero.

Model	Lineage	Seen stacks	Unseen inside	Difference
CodeLlama-7B	Meta	+3.36 [+1.57, +5.27]	−1.92 [−3.70, −0.14]	+5.28 [+3.20, +7.54]
CodeLlama-34B	Meta	+2.27 [+0.44, +4.09]	−0.77 [−2.62, +1.12]	+3.03 [+0.78, +5.28]
Llama-3.1-8B	Meta	+2.30 [+0.47, +4.15]	−1.05 [−2.72, +0.63]	+3.35 [+1.50, +5.30]
StarCoder2-15B	BigCode	+1.50 [−0.02, +3.00]	−1.19 [−2.93, +0.49]	+2.68 [+0.77, +4.65]
Gemma-3-12B	Google	+2.63 [+0.56, +4.64]	−2.09 [−3.81, −0.42]	+4.72 [+2.62, +6.85]
Granite-3.1-8B	IBM	+3.66 [+1.54, +5.68]	−0.24 [−2.02, +1.57]	+3.91 [+1.41, +6.32]
difference significant on				6 of 6 models

Table 1 is the core measurement. Breadth's advantage over clean-code tuning is **2.7 to 5.3 points larger** on stacks built from seen transforms than on stacks containing an unseen family, and the gap is significant on **every one of the six models** we could measure it on, spanning four model lineages. What breadth buys is recombination of what the training data covered, bounded exactly by that coverage.

4.2 The reversal test

The second test is independent of the first and gives the same answer. We evaluate the same adapters, on the same programs, asked backwards, and grade by execution: a prediction is correct if the produced call actually returns the gold value.

Table 2. **The direction ratio: how much of a forward gain survives the task being run backwards.** Forward is output prediction; backward is input prediction on the same programs, graded by execution. No adapter is trained on the backward task, so any backward gain is transferred program understanding rather than task fitting. DR < 0 means the arm bought forward accuracy by *losing* backward competence. Every adaptation method that fits the forward task directly is negative; only anchoring and family exposure are not. Both directions use the same 412 programs and the same six conditions. Bold intervals exclude zero; the ratio is descriptive and is read with its components.

Method	Arm	Forward gain	Backward gain	DR
Format-only control	`formatonly`	+1.47 [+0.90, +2.10]	−0.44 [−0.81, −0.08]	−0.30
Clean-code tuning	`tuned_L0`	+19.34 [+16.87, +21.81]	−1.60 [−3.41, +0.35]	−0.08
Training breadth	`mono_all`	+19.46 [+16.68, +22.01]	−0.89 [−2.86, +1.04]	−0.05
Paired consistency (ours)	`cons_lam3`	+20.88 [+18.07, +23.49]	+0.62 [−1.56, +2.81]	+0.03
Family exposure	`tuned_X1`	+17.35 [+14.98, +19.73]	+1.63 [+0.11, +3.20]	+0.09

Table 2 reports the *direction ratio* — backward gain divided by forward gain, both measured against the untuned model. A negative ratio means the arm bought forward accuracy by *losing* backward competence on the very same programs. **Every method that fits the forward task**

directly is negative: the format-only control at −0.30, clean-code tuning at −0.08, breadth at −0.05. At most a tenth of any forward gain survives the flip, and for these arms the surviving fraction is negative.

4.3 What the two tests agree on

A model that had learned to read obfuscated programs would keep that competence when the program is stacked with something new, and when the question is asked from the other end. These adapters keep neither. Both tests point at the same explanation: what is acquired is competence at *this task on these surfaces*. Section 5 establishes what that surface is.

5 RQ2: What is memorised?

Section 4 established that adaptation does not survive leaving its training distribution, in either direction. This section identifies what it latches onto instead, because that is what determines the fix: if adaptation anchors on something a transform can change, the remedy is to anchor it on something a transform cannot.

Three instruments, sharing no machinery, converge on the same answer.

5.1 The gain tracks identifier surface

Breadth's advantage on a stack depends on the stack still containing an *identifier* transform — renaming, which leaves recognisable token-level residue — rather than on the stack's depth or the number of mechanisms in it.

Table 3. **RQ2: breadth's stack gain depends on an identifier transform being present.** Breadth over the clean-code control, split by whether the stack contains an identifier transform, with the *difference* between the groups. The rule defining the split was fixed before the unseen-containing stimulus existed, which makes the second row an out-of-sample test of it. We report the difference because one interval excluding zero while another does not is not a test of the difference between them.

Stimulus	With identifier	Structural only	Dif
All-seen depth-3/4 stacks (in-sample)	+5.00 **[**+3.02**,** +6.88**]**	+1.61 [−1.02, +4.23]	+3.39 **[**+
Stacks containing the unseen family (out-of-sample)	+1.98 [−0.12, +4.18]	−3.88 **[**−6.08**,** −1.71**]**	+5.85 **[**+

We report this as a contrast rather than an ordering. The rule defining the split (identifier present versus absent) was fixed *before* the unseen-containing stimulus existed, which makes the second row of Table 3 an out-of-sample test of it. The difference is +3.39 on all-seen stacks and grows to +5.85 on stacks containing the unseen family: on the inputs furthest from training, breadth helps only where an identifier surface remains to key on.

5.2 The unit of transfer is a surface, not a mechanism

Splitting the held-out family into its two mechanisms — arithmetic rewriting and string encoding — gives a result that no account based on composable skills survives. The two halves **do not transfer to each other** (+1.49 [−0.95, +3.93] and −0.10 [−2.38, +2.19]), yet **either half alone recovers about 90 %** of the whole adapter's gain on the full family (+4.12 and +4.20 against +4.61). If the adapter had learned "arithmetic skill" and "string skill", each half would deliver roughly half. It does not; each half delivers nearly all of it.

We are deliberate about what this does *not* establish. The natural reading is that what transfers is the scaffolding both halves share. The test that discriminates that reading from a weakest-link

alternative — scoring each half-adapter on programs containing *none* of its own mechanism, the only place a shared-surface account can act alone — returned UNDECIDED on all three of its pre-registered rules. We report that the halves do not decompose, name shared surface as the leading hypothesis, and stop there.

5.3 Adaptation polarises rather than deepens

Breadth's log-probability of the gold answer sits about 4 nats below the clean-code control on *every* condition, and the entire deficit lives in the items it gets *wrong*: where it is right it is *more* confident (−0.16 versus −0.33), where it is wrong it is roughly *twice as far* from the gold (−12.95 versus −7.00). Sharpening is free on conditions the model has seen and expensive on a family where its decisions are more often wrong. This is the mechanism behind the unseen-family tax in Table 1: breadth does not become a worse reader, it becomes a more committed guesser.

5.4 What to anchor to

The three instruments agree: adaptation anchors on the surface a transform leaves behind. That surface is, by construction, exactly what an obfuscator is free to change — which is why competence built on it does not survive a new transform family or a reversed question.

The remedy follows directly. Anchor the model to something the transform *cannot* change: its own behaviour on the unobfuscated parent of the same program. The parent exists for every training row by construction, and it is the one thing a semantics-preserving transform leaves invariant. Section 6 is that objective.

6 RQ3: Anchoring to clean-code behaviour

Section 5 ended with a requirement rather than a proposal: anchor the model to something a semantics-preserving transform cannot change. Exactly one such object is available for every training row, by construction — the model's own behaviour on the unobfuscated parent of the same program. We supervise against it directly:

$$\mathcal{L} = \mathrm{CE}(y \mid x_{\mathrm{obf}}) + \lambda \cdot \mathrm{KL}\big(p_{\mathrm{teacher}}(\cdot \mid x_{\mathrm{L0parent}}) \,\|\, p_{\mathrm{student}}(\cdot \mid x_{\mathrm{obf}})\big) \tag{2}$$

at the answer tokens, where the teacher is a frozen clean-code-tuned adapter reading the unobfuscated parent, and $\lambda = 3$ is a plateau from a sweep on the trainable grid. The student never sees the parent; only the frozen teacher does. Consequently the objective trains on *exactly* the rows breadth trains on — 26,841 rows, 1,257 steps for both — and at $\lambda = 0$ it reduces to breadth identically. Breadth is therefore not merely a comparable baseline but the exact data-matched control, which is what lets the difference between them be read as the objective rather than as extra exposure.

6.1 The ablation is the contribution

The objective's mechanism is best shown by varying the one ingredient the diagnosis in Section 5 says should matter. Replacing the clean-code-tuned teacher with an untuned base model makes the arm **collapse** (−8.98 [−11.44, −6.43] against breadth on the unseen family). Replacing it with the breadth model makes the arm **inherit breadth's tax** (−4.20 [−6.27, −2.14] against the clean-code teacher) while matching on seen conditions. Across all three teachers, the student lands at roughly the teacher's unseen-family level plus one point.

The seen-condition gain therefore comes from the supervised term and is teacher-independent; the unseen-family behaviour is *distilled from the teacher*. The method is "distil from a clean-code-*tuned* model on the clean parent", and we name it as such rather than claiming a new form of invariance.

6.2 What it repairs, and on which models

Table 4. **RQ3: the three findings across eight models and four lineages**, all read from one evaluation phase. √ replicated, ○ inconclusive, × refuted. R1 is breadth's unseen-family tax (`mono_all` − `tuned_L0` on the held-out family); R2 is anchoring removing it (`cons_lam3` − `mono_all`); R3 is anchoring's clean-code cost (`cons_lam3` − `tuned_L0` on clean code). Bold intervals exclude zero.

Model	Lineage	**R1** unseen tax	**R2** anchoring repair	**R3** clean-code cost
CodeLlama-7B	Meta	−3.71 **[−5.93, −1.56]** √	+5.44 **[+3.37, +7.41]** √	−0.54 [−1.98, +1.08] √
CodeLlama-13B	Meta	−4.04 **[−6.26, −1.89]** √	+3.87 **[+2.06, +5.77]** √	−0.72 [−2.28, +0.90] √
CodeLlama-34B	Meta	−2.72 **[−5.02, −0.49]** √	+3.62 **[+1.40, +5.76]** √	+0.48 [−0.84, +1.80] √
Llama-3.1-8B	Meta	−2.72 **[−4.70, −0.91]** √	+3.38 **[+1.57, +5.36]** √	−0.30 [−1.98, +1.26] √
StarCoder2-15B	BigCode	−2.72 **[−4.94, −0.58]** √	+2.31 **[+0.58, +4.03]** √	−0.96 [−2.51, +0.60] √
Gemma-3-12B	Google	−4.70 **[−6.93, −2.55]** √	+2.64 **[+0.91, +4.53]** √	−1.80 **[−3.53, −0.06]** ×
CodeGemma-7B	Google	−1.98 [−3.95, +0.16] ○	−0.41 [−2.39, +1.49] ○	−2.04 **[−3.77, −0.18]** ×
Granite-3.1-8B	IBM	−0.91 [−2.97, +1.15] ○	−1.89 **[−3.54, −0.25]** ×	−3.41 **[−5.39, −1.26]** ×
replicated / inconclusive / refuted		6 / 2 / 0	6 / 1 / 1	5 / 0 / 3

Table 4 reads all eight models from a single evaluation phase. Before reading it we recomputed each previously published contrast through the same configuration and required the published point to fall inside the new interval; all four incumbents pass, with a largest disagreement of 0.85 points, so a departure below is a property of the model rather than of the measurement.

Three readings, at the strength the panel supports:

- **Breadth's unseen-family tax is the robust finding.** Its sign is the same on *every* model — six significantly, none contradicting — across four lineages and 7B–34B.
- **The repair holds on six of eight models and reverses on one.** On Granite-3.1-8B the anchored arm is *worse* than breadth on the unseen family. We name that model rather than relegating it.
- **The clean-code claim is conditional.** "Anchoring pays no clean-code cost" holds on five of eight. It was discovered on four models of one lineage and holds on all four of them.

Every failure above falls outside the lineage the findings were discovered on, and all three are four-of-four inside it. We report that as a *methodological* warning and not as a claim about lineages: Fisher's exact on the sharpest split gives $p = 0.0714$ one-sided with $n = 8$ models, and the two models of one vendor account for two of the three failures of the same finding, so vendor and lineage are confounded. What it does establish is that single-lineage evaluation would have hidden all of it.

7 RQ4: Does the anchored model generalise?

Section 4 committed to two tests before any method was on the table. This section runs the same two on the anchored model. If the diagnosis was right — that adaptation fails because it anchors on a surface the transform can change — then anchoring to something the transform cannot change should move the outcome of both tests, and it should do so *specifically* where the input has left the training distribution.

7.1 The composition test, re-run

Table 5 is the divergence ladder: stacks ordered by how far they sit from what breadth was trained on, from all-seen depth-2 composites (d0) to stacks containing a family the model has never seen

Table 5. **RQ4: the divergence ladder.** Composites that contain a transform family the model has never seen. Breadth's advantage flips sign the moment such a component enters, while anchoring rises *above* the clean-code control. The pre-registered rule required both that anchoring hold and that breadth fall below the control; only the first is established, since the pooled interval at d2 straddles zero. CodeLlama-7B, 287 programs common to all six composites, fixed before any system was evaluated.

Level	Stimulus	mono−clean	cons−clean	cons−mono
d0	depth 2, all seen	+3.49 [+2.04, +5.01]	–	+0.87 [−0.59, +2.33]
d1	depth 3–4, all seen	+4.15 [+2.37, +5.91]	+5.02 [+3.41, +6.69]	+0.87 [−0.59, +2.33]
d2	depth 2, **unseen inside**	−1.71 [−3.60, +0.31]	+2.13 [+0.66, +3.60]	+3.84 [+1.90, +5.70]
d3	depth 3, **unseen inside**	−0.35 [−2.67, +1.86]	+3.26 [+0.81, +5.92]	+3.60 [+1.40, +5.81]

(d2, d3). The stimulus at d2 and d3 did not exist before this work; every composite in the prior literature, and in our own earlier experiments, is built from transforms the model has seen.

Two things happen once an unseen component enters the stack. Breadth's advantage over the clean-code control **flips sign**, from +3.49 and +4.15 on seen stacks to −1.71 and −0.35. And anchoring, which merely *matches* breadth on seen stacks (+0.87 [−0.59, +2.33] pooled), rises **above the clean-code control** (+2.13, +3.26) and **above breadth** (+3.84, +3.60). The anchored model's advantage appears only where the input diverges from training, which is the signature the diagnosis predicted and the opposite of what a better surface heuristic would show.

The pre-registered verdict, stated first. We fixed the decision rule and both readings of the outcome before any cell existed. The rule required *both* that anchoring hold *and* that breadth fall below the clean-code control; only the first is established, because the pooled interval at d2 spans zero. By that rule the outcome is UNDECIDED, and we report it as such before the richer reading.

Breadth's collapse is not uniform. The pooled d2 figure averages significantly negative cells with a significantly positive one. Split by the identifier rule of Section 5 — committed 91 minutes before the first d2 cell was written — the identifier-containing stacks sit at +1.98 [−0.12, +4.18] and the structural-containing stacks at −3.88 [−6.08, −1.71]. Breadth is not broken by an unseen component as such; it is broken where the stack leaves it no identifier surface to key on. The mechanism from Section 5 holds on stimulus it was never fit to.

Table 6. **RQ4: the divergence result across the panel**, on stacks containing the unseen family. cons−mono is anchoring's advantage over breadth; fam is an arm trained on the unseen family's trainable sibling, included because family exposure is the largest effect in the table and replicates on every model. Bold intervals exclude zero.

Model	Lineage	cons−mono	cons−clean	fam−clean
CodeLlama-7B	Meta	+3.84 [+1.90, +5.70]	+2.13 [+0.66, +3.60]	+5.93 [+4.10, +7.78]
CodeLlama-13B	Meta	+3.22 [+1.74, +4.65]	+2.02 [+0.58, +3.53]	+5.97 [+4.11, +7.91]
CodeLlama-34B	Meta	+3.95 [+2.13, +5.81]	+3.10 [+1.63, +4.58]	+4.19 [+2.48, +5.94]
Llama-3.1-8B	Meta	+2.87 [+1.20, +4.62]	+1.86 [+0.27, +3.53]	+6.67 [+4.58, +8.71]
StarCoder2-15B	BigCode	+1.36 [−0.08, +2.71]	+0.35 [−1.13, +1.86]	+3.64 [+1.90, +5.35]
Gemma-3-12B	Google	+2.44 [+0.97, +3.99]	−0.08 [−1.67, +1.51]	+5.35 [+3.21, +7.37]
CodeGemma-7B	Google	−1.59 [−3.26, +0.12]	−2.40 [−4.11, −0.85]	+5.23 [+3.25, +7.21]
Granite-3.1-8B	IBM	−1.47 [−2.99, +0.00]	−1.86 [−3.80, +0.12]	+7.29 [+5.19, +9.65]

Table 6 runs the same read across the panel. The anchoring advantage on unseen-containing stacks replicates on five of eight models, is inconclusive on three, and is refuted on none. The three inconclusive models are the three whose ladder-level benefit was itself weakest or negative, and on Granite — the model where the repair reverses — the advantage is negative here too. Family exposure, by contrast, is significant on all eight and is the largest single effect in the table.

7.2 The reversal test, re-run

Returning to Table 2: every adaptation method that fits the forward task directly has a negative direction ratio. Anchoring's is +0.03. It is the only arm in the supervised-fine-tuning family whose forward gain cost nothing backwards, and its backward accuracy is the best of that family.

We state precisely what that does and does not mean. Anchoring does **not** acquire backward competence: against the untuned model it is +0.62 [−1.56, +2.81] backwards — undamaged, not improved. The claim the data supports is that *every existing method pays backward competence for its forward gain, and anchoring is the first that does not.* The one arm in the panel that gains in both directions is the one trained on the unseen family's sibling (+0.09; +1.63 [+0.11, +3.20] backwards). Exposure to a *family* transfers in a way exposure to *transforms* does not, which is the memorisation account stated from the other side.

7.3 The bound

We state it in this section rather than leave a reviewer to extract it. On the unseen family *alone*, the anchored model is as robust as clean-code tuning and no more: −0.30 at 7B, −0.33 at 13B, +0.74 at 34B against the control, seventh of thirty-three arms on that column. Every arm that beats it there was trained on that family's sibling. Combined with the teacher ablation, the correct statement is: anchoring buys breadth's recombination benefit at little cost, extends it to composed inputs the model has never seen, and stops paying the backward price every other method pays — and it does not add the capability breadth lacks. It is a repair of what adaptation breaks, not a route to invariance.

8 Related Work

This paper joins three distinct conversations in the literature. First, we differentiate from the deobfuscation (DOBF) lineage by strictly evaluating code that remains obfuscated, measuring semantic competence under transformation rather than recovery capability. Second, in the domain of model merging and modular adaptation, we contribute a negative result supported by a highly rigorous random-gate control, warning that task-vector geometry is often dominated by initialisation rather than genuine knowledge. Finally, we contribute to the growing literature on robustness to semantics-preserving transformations and direction specificity in code understanding.

9 Threats to Validity

The held-out family is a proxy. Every unseen-family number in this paper is measured on X1, a trainable sibling of the quarantined family H1, whose evaluation budget was spent under a pre-registered two-pass discipline. The sibling reproduces H1's difficulty at $r = 0.9992$ with a mean absolute difference of 0.48 points on arms trained on neither, and transfer between them was lossless (0.08 points) for arms trained on the sibling. That calibration is a result and is reported as one, but it is a single family pair, and a second hard family with a different surface is the experiment that would settle whether "family" means "surface family".

Coverage is not uniform, and contrasts are paired. Structural transforms decline on some programs by design and the held-out family requires a minimum number of sites, so conditions cover different

program sets. Every contrast runs on a subset recorded before any system was evaluated, and where two groups are compared the program set is intersected first. Where that intersection moves an estimate we report both; on the composition test it moves one model's unseen-side level from non-significant to significant, and the paired difference is unaffected.

Eight models is a small sample for a claim about models. Section 6 declines to make one. Every finding replicates uniformly on the lineage it was discovered on and unevenly off it, and with $n = 8$ and vendor confounded with lineage that supports a warning about single-lineage evaluation, not a result about lineages. The panel's strongest non-lineage model is also the one an untuned capability screen scored at zero and would have discarded; removing it would have made the lineage pattern look cleaner and be more wrong.

The reversal test is one model deep. The direction ratio is measured on CodeLlama-7B; the panel-wide backward grids are in flight and are not reported here.

Instrument failures we report rather than omit. The causal attention instrument is inert: an identifier knockout moves the gold log-probability by under 0.5 % for every arm on every condition, and both of its pre-registered hypotheses were refuted. The attention account is correlational and is labelled as such. Separately, the test that would discriminate the shared-surface reading in Section 5 from a weakest-link alternative returned undecided on all three of its rules; we report the leading hypothesis as a hypothesis.

Single language. Every headline is Python. The JavaScript corpus exists and a cross-language replication runs on it, but the toolchain to regenerate it is absent on our cluster, so no new JavaScript condition could be built.

10 Data Availability

Data/code is available in project's website [].

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[2] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A Taxonomy of Obfuscating Transformations. *Technical Report* (1997).

[3] Christian Collberg, Clark Thomborson, and Douglas Low. 1998. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) *(POPL '98)*. Association for Computing Machinery, New York, NY, USA, 184–196. doi:10.1145/268946.268962

[4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[5] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie LIU, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCode{BERT}: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*. https://openreview.net/forum?id=jLoC4ez43PZ

[6] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158

[7] Anh H. N. Nguyen, Jack Le, Ilse Lahnstein Coronado, and Tien N. Nguyen. 2026. The Effect of Code Obfuscation on Human Program Comprehension. arXiv:2603.07668 [cs.SE] https://arxiv.org/abs/2603.07668

[8] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. arXiv:2203.13474 [cs.LG] https://arxiv.org/abs/2203.13474

[9] Serge Lionel Nikiema, Jordan Samhi, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. The Code Barrier: What LLMs Actually Understand? arXiv:2504.10557 [cs.SE] https://arxiv.org/abs/2504.10557

[10] Philip O'Kane, Sakir Sezer, and Kieran McLaughlin. 2011. Obfuscation: The Hidden Malware. *IEEE Security and Privacy* 9, 5 (2011), 41–47. doi:10.1109/MSP.2011.98

[11] Daniel Votipka, Seth Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. 2019. An Observational Investigation of Reverse Engineers' Process and Mental Models. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) *(CHI EA '19)*. Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3290607.3313040